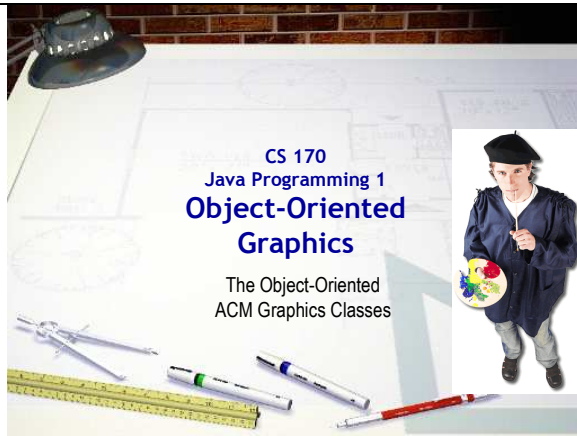




Slide 1

CS 170

Java Programming 1

Object-Oriented

Graphics

Duration: 00:00:18

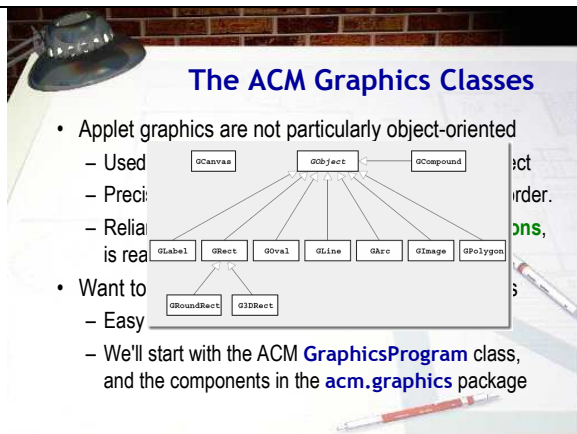
Advance mode: Auto

Notes:

Hello, Welcome to the CS 170, Java Programming 1 Lecture on Object-Oriented Graphics. In this lecture, you'll learn to use the graphics classes that are part of the ACM class libraries, that you first met several weeks ago. The ACM class libraries were developed by the ACM Java Task Force in Fall 2005 as a "a stable collection of pedagogical resources designed to make it easier to teach Java to first-year computing students without having those students overwhelmed by its complexity."

I'd also like to take a minute to mention the ACM once again. The ACM (which stands for the kind of weird name, Association of Computing Machinery), is the professional organization for computing professionals. It's an international scientific and educational organization dedicated to advancing the arts, sciences, and applications of information technology.

For students, an ACM membership is an incredibly good deal. It costs about \$20 a year and you get access to a wealth of online materials, including more than 500 online books and 3,000 online courses. To sign up or just look around, visit the ACM Web site at <http://www.acm.org>.



Slide 2

The ACM Graphics Classes

Duration: 00:00:53

Advance mode: Auto

Notes:

Last week, you learned how to use Java's Graphics class to draw ovals, rectangles, and text, and used that knowledge to draw some simple pictures.

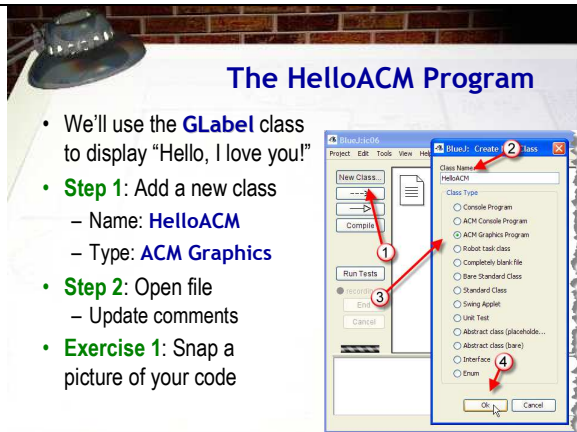
While the drawing methods you learned in that unit—drawRect(), fillOval(), and so on—were simple, they weren't particularly object-oriented. Instead of dealing with oval objects that know how to draw themselves, you "issued orders", giving precise instructions for completing your drawings. If you look back on your Self-Portrait class, for instance, you'll see that it mostly consists of a sequence of commands, that have to be carried out in a particular order.

Although we did use a Graphics object to carry out the work, we also gave precise instructions on how to do the task, and in what order. This reliance on a specific sequence, and the focus on actions, is really at the heart of procedural programming.

Now, though, it's time for us to start moving away from this style of programming, and to start working with independent graphical objects. While the built-in Java class libraries provide support for creating your own graphical objects, they don't really have any easy-to-use components. So, for this lesson we'll use the ACM GraphicsProgram class, and the graphical shapes available in the acm.graphics package, shown in this illustration.

We'll start by using the GLabel class to display the traditional "Hi Mom" program. Then, during the semester, we'll use many of the other classes to illustrate different object-oriented

techniques and principles.



The HelloACM Program

- We'll use the **GLabel** class to display "Hello, I love you!"
- Step 1:** Add a new class
 - Name: **HelloACM**
 - Type: **ACM Graphics**
- Step 2:** Open file
 - Update comments
- Exercise 1:** Snap a picture of your code

Slide 3

The HelloACM Program

Duration: 00:00:49
Advance mode: Auto

Notes:

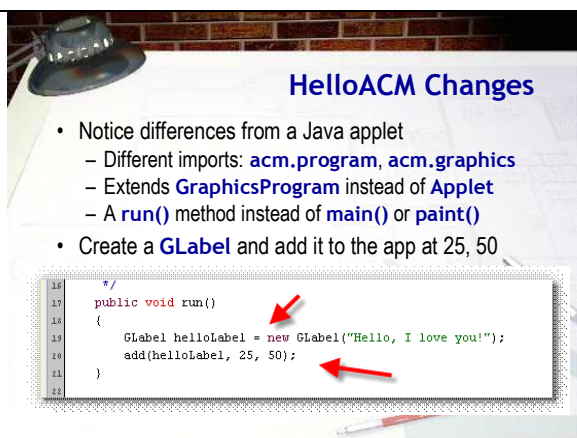
The OCC version of BlueJ automates the process of creating an ACM GraphicsProgram, just like the other ACM programs. We'll get started by using the simplest graphical component, named GLabel, and asking it to display "Hello, I love you!", pretty much like we did with our first graphical applet.

To do that just, go ahead and open BlueJ, starting a new project for this lecture, if you like. Then:

1. Click the New Class button to add a new class to your project
2. Name the class HelloACM
3. Click the radio button for ACM Graphics when asked to select the type
4. Finally, click the OK button.

Once the class has been created, open it and update the comments at the top of the file.

Open the lab exercise document for this week, create a new section for this lecture, and shoot me a screen-shot showing the file open in your editor with the comments updated.



HelloACM Changes

- Notice differences from a Java applet
 - Different imports: **acm.program**, **acm.graphics**
 - Extends **GraphicsProgram** instead of **Applet**
 - A **run()** method instead of **main()** or **paint()**
- Create a **GLabel** and add it to the app at 25, 50

Slide 4

HelloACM Changes

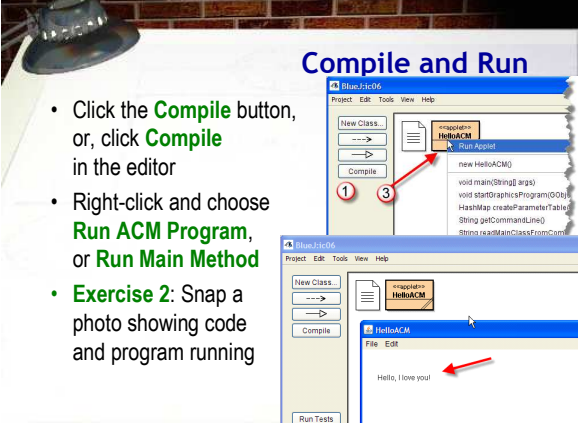
Duration: 00:00:41
Advance mode: Auto

Notes:

Once you've added your comments, let's take a quick look at the rest of the code.

1. First, notice that instead of importing the `java.applet` and `java.awt` packages, we import `acm.program` (which contains the `ConsoleProgram` as well as the `GraphicsProgram` classes) and `acm.graphics`, which contains the object-oriented graphical components we're going to use. If you want to use the `Color` or `Font` classes, (which we'll do in a bit), you'll also need to manually add the `java.awt` package where those classes are found.
2. Next, notice that instead of extending the `Applet` class, `HelloACM` extends the `GraphicsProgram` class. `GraphicsProgram` is actually subclass of `Applet`, so, in a way, we're still extending the `Applet` class.
3. Finally, instead of a `paint()` or `main()` method, you have a method named `run()`, just like the `ACM ConsolePrograms`

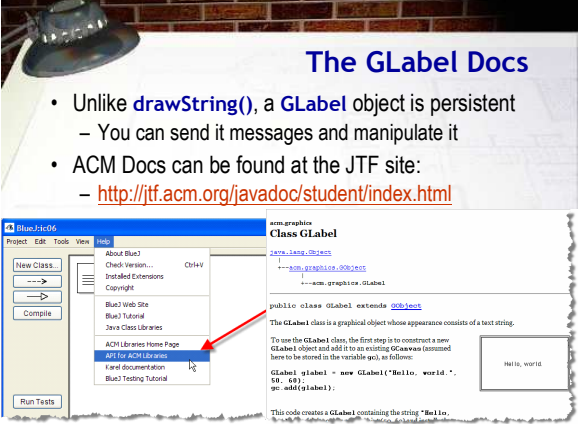
	you've written. Now, let's add some code to the run() method. The class we'll use to display text in our ACM graphical programs is named GLabel. You create a GLabel object using the same kind of syntax you'd use to create any Java object: 1. Create an object variable. In the picture you can see that I've named my variable helloLabel. The Label at the end helps remind me that it's a "label" object. 2. Initialize the object by using the new operator and a constructor. We pass the GLabel constructor the words we want it to "manage" for us: "Hello, I love you". Once we've created our graphical object, we need to add it to the surface of your window. To do that, we use the GraphicsProgram method named add(), passing it our object, and the position that we want the label to appear at. (We could also just pass it our label, and then use the setLocation() method to position the label in a separate step.)
--	---



Slide 5
Compile and Run
 Duration: 00:00:31
 Advance mode: Auto

- Click the **Compile** button, or, click **Compile** in the editor
- Right-click and choose **Run ACM Program**, or **Run Main Method**
- **Exercise 2:** Snap a photo showing code and program running

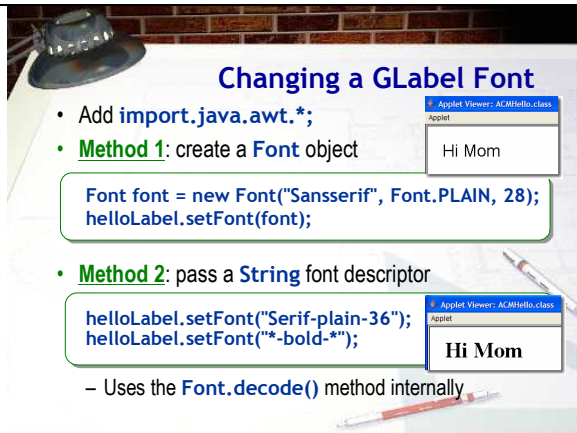
Notes: Once you've added the two lines to your run() method, click the Compile button (as shown in the screenshot here). If there are any errors, fix them in the editor and recompile. Repeat the process until you get a "clean compile". To run your program, right-click and choose Run ACM Program or Run Main Method from the context menu. Run Applet also works, but not as well, especially when it comes time to use images, which we'll do shortly. Once you have the program compiled and running, shoot me two screen-shots: 1 of the source code in your run method, and another of the output as your program actually runs, like that shown here.
--



Slide 6
The GLabel Docs
 Duration: 00:00:37

- Unlike **drawString()**, a **GLabel** object is persistent
 - You can send it messages and manipulate it
- ACM Docs can be found at the JTF site:
 - <http://jtf.acm.org/javadoc/student/index.html>

Notes: OK, I admit, the HelloACM program doesn't look any more impressive than the HelloApplet program, where we used Java's drawString() method to do pretty much the same thing. The difference, though, is that helloLabel is an object that you can send messages to and manipulate on the screen. When you use drawString() all you have are some pixels painted on the display. To see what kinds of things you can do with GLabel objects, like helloLabel, head on over to the ACM Library documentation site, and locate and click the link for the GLabel class. Just as with the Karel docs, you can get to the ACM Library docs by using the special link on the BlueJ Help menu.
--



Changing a GLabel Font

- Add `import java.awt.*;`
- **Method 1:** create a `Font` object


```
Font font = new Font("Sansserif", Font.PLAIN, 28);
helloLabel.setFont(font);
```
- **Method 2:** pass a `String` font descriptor


```
helloLabel.setFont("Serif-plain-36");
helloLabel.setFont("*-bold-*");
```

 - Uses the `Font.decode()` method internally

Slide 7

Changing a GLabel Font

Duration: 00:00:53

Advance mode: Auto

Notes:

There are three things you can do to change the appearance of the labels you create:

- change the font used by the label.
- change the color that the words are displayed in.
- change the words themselves.

Each of these requires you to use a mutator method that changes the state of your label object. Let's look briefly at each of them. Before we do that though, add the line:

```
import java.awt.*;
```

along with the other imports to the top of your program.

OK, now let's look at changing the font.

You can change the font used by the label by creating a `Font` object (just like we did in that last mini-lecture), and then calling your label's `setFont()` method, like this:

```
Font mediumFont = new Font("Sansserif",
    Font.PLAIN, 28);
helloLbl.setFont(mediumFont);
```

This displays your greeting in a 28-point Arial or Helvetica-style font so your program looks like the screenshot shown to the right.

You can also use the `setFont()` method with a `String` font descriptor, instead of a separate `Font` object when you call `setFont()`. To do this, your `String` needs to be in the form: "family-style-size". For the style, instead of using constants, like `Font.PLAIN`, just use the words `bold`, `italic`, `plain`, or `bolditalic` (all one word). The style portion is not case-sensitive, though, so you can use `BOLD` or `Bold` if you like.

If you want to leave one of the three parts of your font "as-is", then just use an asterisk, like this: `helloLbl.setFont("*-bold-*");`. Internally, this version of `setFont()` uses Java's `Font.decode()` method to create an unnamed `Font` object from the `String` you pass as an argument.



Changing Color and Caption

- Change the color by using the `setColor()` method


```
helloLabel.setColor(Color.red);
```

 - You can't change the background color
- Change the caption with the `setLabel()` method


```
helloLabel.setLabel("Hello Muddah");
```
- **Exercise 3:** change the font & color, and change the message to add your name. Run & snap

Slide 8

Notes:

You can change the color for the text that's displayed by sending the `setColor()` message to the label, like this:

```
helloLabel.setColor(Color.red);
```

Of course, you can use a custom `Color` object, and not just the constant that I used here. Unlike many of the other ACM Graphics objects, you can't change the background color, only the foreground.

You change the message that the label displays by providing your object with a different `String` using the `setLabel()` method. Here's a piece of code that changes the message to represent the opening line from Alan Sherman's classic novelty song

Changing Color and Caption

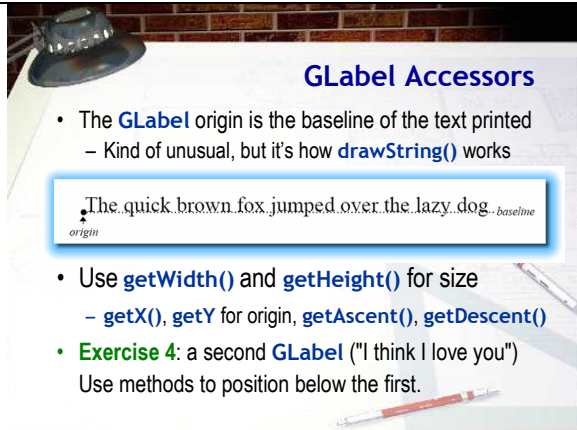
Duration: 00:00:47

Advance mode: Auto

about a little boy stuck at summer camp and the letter he sends to his parents:

```
helloLabel.setLabel("Hello Muddah");
```

OK, now it's your turn. For Exercise 3, change the font and color, and then change the message by adding your name so that it says: "Hello, my name is Steve." replacing my name with yours, of course. Run the program and snap me a picture.



- The **GLabel** origin is the baseline of the text printed
 - Kind of unusual, but it's how **drawString()** works
- Use **getWidth()** and **getHeight()** for size
 - **getX()**, **getY** for origin, **getAscent()**, **getDescent()**
- **Exercise 4:** a second **GLabel** ("I think I love you")
Use methods to position below the first.

Slide 9

GLabel Accessors

Duration: 00:00:37

Advance mode: Auto

Notes:

In the HelloACM program, we positioned the GLabel object, `helloLbl`, by supplying two additional arguments representing the horizontal, or *x* location (measured from the left-hand side of the program's window), and the vertical, or *y* location, measured from the top of the window when we added the label to the program.

Most of the components in the `acm.graphics` package use similar *x,y* coordinates to position themselves, using the upper-left corner of the component as the component origin. The GLabel class acts a little differently. Instead of placing the component's upper-left corner as the origin, GLabel uses the left-hand side of the text and the baseline of the text. This was done so that printing using GLabel would match the scheme adopted for the `drawString()` method, as shown in this figure.

One difficulty with using `drawString()` is that there is no easy way to find out how wide or tall a message is so you can figure out where to place it. Your first applet exercise, you recall, was using `drawString` to center two lines of text in an applet. The only way to do it was to use trial and error for the *x,y* values. With GLabel, you have several methods that will help you place your text exactly where you want.

First, you can find out how much space the label takes by using its `getWidth()` and `getHeight()` methods. To find the origin of your GLabel object, you can use the methods `getX()` and `getY()`, and then, you can also find out if any letters in your label are "hanging down" below the baseline (like a lowercase *p*, *q*, or *j* would do), by calling the method `getDescent()`. Similarly, you can find the height of the tallest character above the baseline using the `getAscent()` method.

Let's see if you can figure out how these work. Add a second GLabel that says, "I think I love you" to your program. Position the second label immediately below the first by using any or all of these methods. Remember that you position your label by supply *x* and *y* values when you call the `add()` method. You'll get those *x* and *y* values by using the accessors from your first label as well as at least one accessor from the second label. You'll have to think a little about the calculations. Run it and shoot me a screen-shot. Your new label can use the default font and color, so it will look different than your first label.

Centering Text

- Centering text was hard with `drawString()`. It's easier with the `GLabel` class
 - Change position using `setLocation()`

```
double cx = this.getWidth(); // or omit the word this
double cy = getHeight();
double x = cx - helloLabel.getWidth() / 2;
double y = cy + helloLabel.getHeight() / 4;
helloLabel.setLocation(x, y);
```

- Exercise 5:** center both lines in your program. Snap a picture.



Slide 10
Centering Text
 Duration: 00:00:56
 Advance mode: Auto

Notes:

One thing you may want to do, is to center a label or other graphical component. Conceptually, and arithmetically, this is easy to do with any of the ACM graphics classes. After creating the object, you'll change it's position using the `setLocation()` method, and then use this simple algorithm or recipe:

- Divide the width and height of the program window in half. Get the width and height by using the program's `getWidth()` and `getHeight()` methods. Notice that you can use the keyword `this` to make it explicit that you are sending the message to your program object, or, you can omit it like I've done in the second line. This gives you the center of the screen, represented by the variables `cx` and `cy`.
- Divide the width of the `GLabel` object in half as well. You use the `getWidth()` method again, but this time, send the message to the label object instead of the program. Subtract half the label width from the center point for the `x` argument.

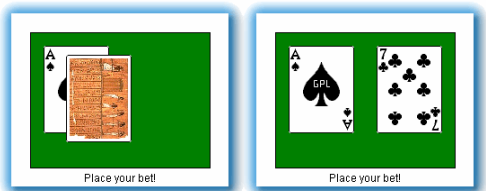
The `y` argument is a little harder. What seems to work best visually is to add one quarter of the label height to the center point for the `y` argument. Note that this means the `y` argument will always be below the window center point, because the label's origin is at the baseline, not the upper-left corner of the text.

Here's the phrase from the previous slides centered horizontally and vertically. I've also added some lines going through the center of the program window and immediately above and below the text.

For Exercise 5, why don't you try centering both lines in your program. Your code will be a little different than what I have here, since you have two lines, and most likely, the top line will be taller (since you changed the font). That means you'll need to add the heights of both lines together before calculating where the bottoms of each line needs to go. Run your program and shoot me a screen-shot.

The TwoCards Applet

- Let's sample some of the other ACM graphics classes
 - Create a new class called `TwoCards`
 - Follow steps from `HelloACM`



Notes:

To give you a quick flavor for some of the other classes in the `acm.graphics` package, let's build a simple program called `TwoCards`. Our program will:

- Display a dark-green "table" in the center of the application with a caption saying "Place your bets!" underneath it.
- Draw two playing cards on the table, one face up, and one face down.
- Flip over the face-down card and set it next to the face-up card when it's clicked with the mouse. You'll learn how to

Slide 11

The TwoCards Applet

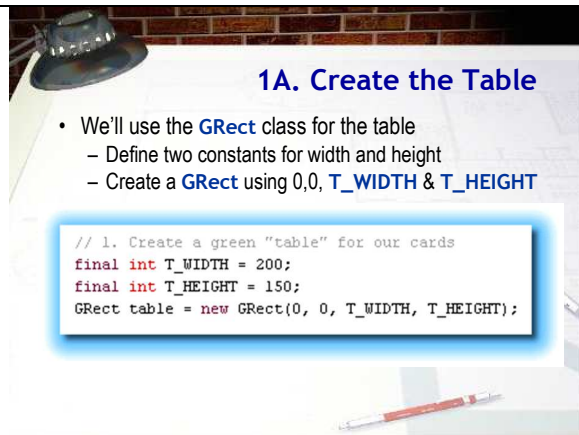
Duration: 00:00:25

Advance mode: Auto

handle mouse-clicks and flip the card in later lessons this week.

Here's a screen shot that shows the program when it first loads (on the left), and a shot of the program after the user has clicked the face-down card:

To start, create an ACM GraphicsProgram named TwoCards, following the same steps you followed for HelloACM earlier in this lesson.



1A. Create the Table

- We'll use the **GRect** class for the table
 - Define two constants for width and height
 - Create a **GRect** using 0,0, **T_WIDTH** & **T_HEIGHT**

```
// 1. Create a green "table" for our cards
final int T_WIDTH = 200;
final int T_HEIGHT = 150;
GRect table = new GRect(0, 0, T_WIDTH, T_HEIGHT);
```

Notes:

To create the table, we'll use the GRect class. GRect is one of the ACM's Fillable classes, so you can specify an interior color that's different than the background. We'll make our interior dark green.

To create the table, start by defining a couple of constants, T_WIDTH and T_HEIGHT to represent its width and height (instead of using literals as magic numbers).

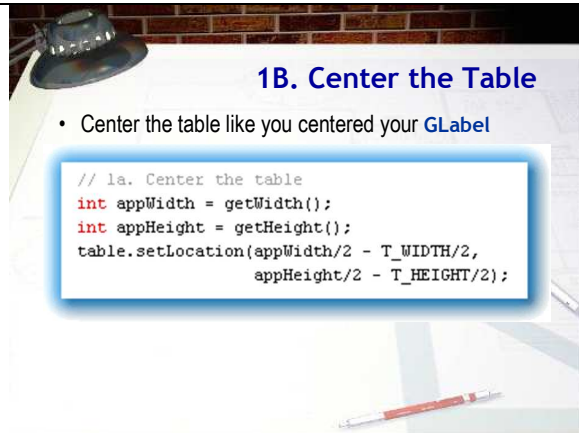
Then, create a GRect object named table using the constructor. Use 0,0 for the origin, and pass your two constants for the width and height like that shown in the screenshot.

Slide 12

1A. Create the Table

Duration: 00:00:25

Advance mode: Auto



1B. Center the Table

- Center the table like you centered your **GLabel**

```
// 1a. Center the table
int appWidth = getWidth();
int appHeight = getHeight();
table.setLocation(appWidth/2 - T_WIDTH/2,
                 appHeight/2 - T_HEIGHT/2);
```

Notes:

To position the table, you'll need to subtract one-half its width and one-half its height from the width and height of your program itself. Remember, you can get those numbers by calling the getWidth() and getHeight() methods.

Then, use the setLocation() method to position the table object in the center of the screen, like that shown in this screenshot.

Slide 13

1B. Center the Table

Duration: 00:00:17

Advance mode: Auto



1C. Color the Table

- **GRect** is a "fillable" class—has optional background
 - Specify the color using **setFillColor()**
 - Turn filling "on" using **setFilled()**
 - Add the table to the surface of the app

```
// 1b. Fill it with a dark green color
table.setFillColor(new Color(0, 128, 0));
table.setFilled(true);
add(table);
```

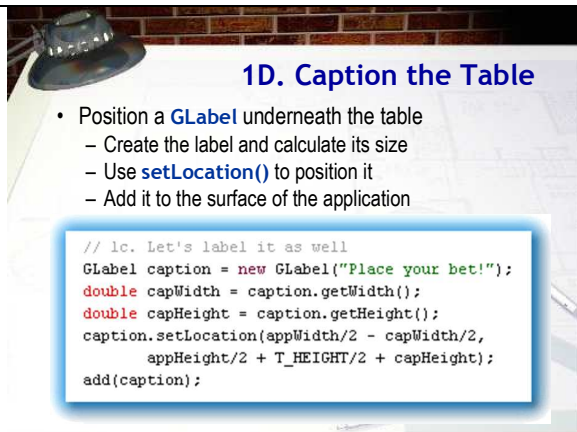
Slide 14
1C. Color the Table
 Duration: 00:00:31
 Advance mode: Auto

Notes:

To color our table, we need to do three things:

- Use the GRect method **setFillColor()** to the background color we want for the table. I've used the Color constructor to create a dark green felt color.
- Send the **setFilled(true)** message to the table to tell it to fill itself in when it's drawn.
- Add the finished component to our program's canvas, so we can see it.

The screenshot shows the code for each of these steps.



1D. Caption the Table

- Position a **GLabel** underneath the table
 - Create the label and calculate its size
 - Use **setLocation()** to position it
 - Add it to the surface of the application

```
// 1c. Let's label it as well
GLabel caption = new GLabel("Place your bet!");
double capWidth = caption.getWidth();
double capHeight = caption.getHeight();
caption.setLocation(appWidth/2 - capWidth/2,
    appHeight/2 + T_HEIGHT/2 + capHeight);
add(caption);
```

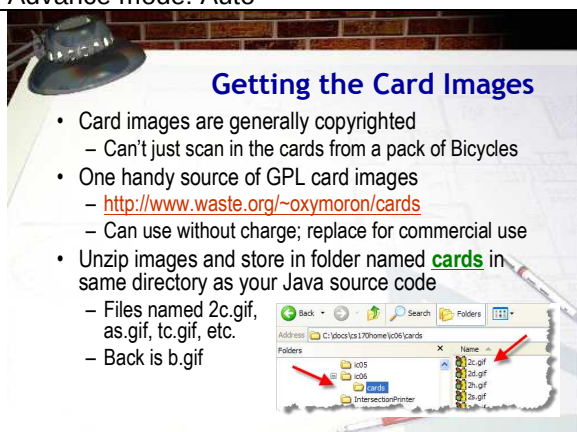
Slide 15
1D. Caption the Table
 Duration: 00:00:31
 Advance mode: Auto

Notes:

Since we've already had experience using the GLabel class, you should be able to figure out how to add the table caption so that its centered right underneath the table itself.

The secret lies in figuring out where the bottom of the table is on the screen, and then adding the height of the caption itself to that number for the vertical position of the label.

The screenshot to here shows what the calculations should look like.



Getting the Card Images

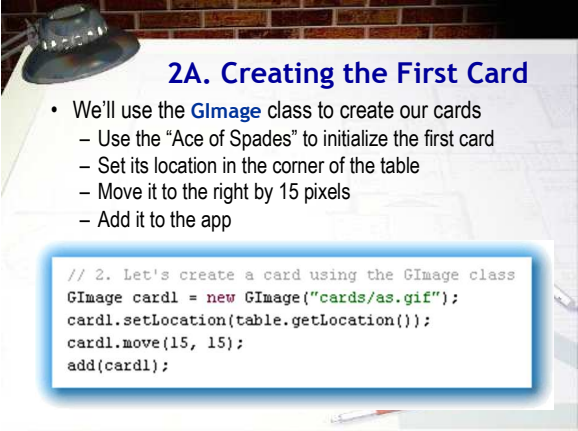
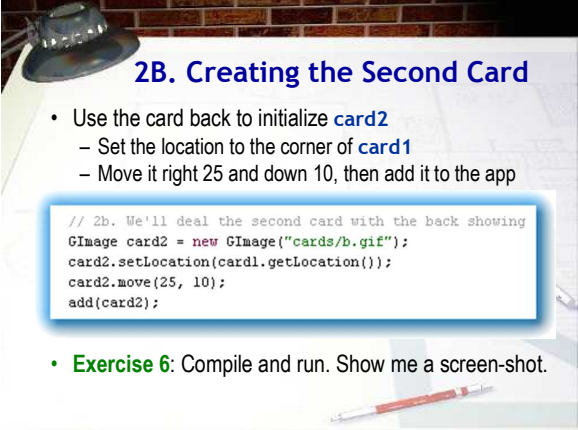
- Card images are generally copyrighted
 - Can't just scan in the cards from a pack of Bicycles
- One handy source of GPL card images
 - <http://www.waste.org/~oxymoron/cards>
 - Can use without charge; replace for commercial use
- Unzip images and store in folder named **cards** in same directory as your Java source code
 - Files named 2c.gif, as.gif, tc.gif, etc.
 - Back is b.gif

Slide 16
Getting the Card Images
 Duration: 00:00:55
 Advance mode: Auto

Notes:

The ACM GImage class allows you to easily display and position objects created from standard JPEG and GIF images. The problem is, where do you find card images? You can't just scan in a pack of Bicycles or screen-capture the back of Windows Solitaire, because the images themselves are copyrighted. One solution is to use some GPL or General Public License software or images. For this problem, I've downloaded the GPL card images from <http://www.waste.org/~oxymoron/cards/>. You can use these images without charge, but if you eventually create a commercial product, you'll probably want to replace them with a different set.

Storing the card images in a folder. When you download the images, store them in a folder named cards located in the same folder as your Java source code. If you unzip them to the same folder where your Java source code is located, this should happen automatically. If you don't have an "un-zip" program, just open a command window and use the magic

	incantation jar xvf cards20.zip. This uses the Java archive tool that is installed with the JDK. I usually use jar instead of Windows XP's built-in zip support. Each card image is stored in a separate file, so they're fairly easy to manipulate. the two of Clubs is in the file 2c.gif, the Ace of Spades is in as.gif, and so on. (To make sure all of the filenames are only two characters long, the "tens" are in a set of files named like tc.gif instead of 10c.gif. Also, the file b.gif is used for the backs of the cards.
 2A. Creating the First Card - We'll use the <code>GImage</code> class to create our cards - Use the "Ace of Spades" to initialize the first card - Set its location in the corner of the table - Move it to the right by 15 pixels - Add it to the app <pre>// 2. Let's create a card using the GImage class GImage card1 = new GImage("cards/as.gif"); card1.setLocation(table.getLocation()); card1.move(15, 15); add(card1);</pre> Slide 17 2A. Creating the First Card Duration: 00:00:37 Advance mode: Auto	Notes: We'll need to create two cards for our program, so I'll just name them card1 and card2. To create the card objects, we'll use the <code>GImage</code> class. <code>GImage</code> actually hides a lot of complexity that you'll be exposed to later (such as locating the image file and loading it into memory). Right now, to construct your first card object, simply pass a String containing the path to the image file you want to use to the <code>GImage</code> constructor. This path needs to be relative to the location of your Java program. In my example, that's "cards/as.gif". Once you've created the card1 object, you can position it with <code>setLocation()</code>. Rather than calculating the location independently, we can just "put it on the table" by asking the table object for its location (using <code>table.getLocation()</code>), and passing that value. Then, because we don't want to display the card in the upper-left corner of the table, we can use the <code>move()</code> method to move the card in from the left and down from the top. Finally, you'll need to add the card to the program, just like any other ACM graphical component. The screenshot here shows the code required to do this.
 2B. Creating the Second Card - Use the card back to initialize <code>card2</code> - Set the location to the corner of <code>card1</code> - Move it right 25 and down 10, then add it to the app <pre>// 2b. We'll deal the second card with the back showing GImage card2 = new GImage("cards/b.gif"); card2.setLocation(card1.getLocation()); card2.move(25, 10); add(card2);</pre> Exercise 6: Compile and run. Show me a screen-shot. Slide 18 2B. Creating the Second Card Duration: 00:00:31 Advance mode: Auto	Notes: The second card uses almost exactly the same code, with the following differences: - We used the image for the back of the card, so the card appears dealt "face down". - We used the location of the first card when positioning our second, rather than the location of the table. - We moved the second card so that a little more of the first card shows. In this case we'll move it right by 25 pixels and down by 10. The code for the second card is shown in the screenshot here. Well, that's enough for this lesson. For Exercise 6, compile and run the <code>TwoCards</code> program. Remember, since it's an ACM Graphics program, you'll run it by choosing Run Main Method.

	You may have problems loading the images if you try to run it as an applet. If you see a red error message that says something like "permission denied", make sure you aren't running as an applet.
--	--